Evolutionary Computation

Report 1

Jędrzej Pluta (160327)
Filip Podalak (160311)

1 Problem Description

We are given a set of n nodes $V = \{0, 1, \ldots, n-1\}$ in a two-dimensional Euclidean plane, loaded from instances `TSPA.csv` and `TSPB.csv`. Each node $i \in V$ is defined by three attributes: its coordinates (x_i, y_i) and a non-negative integer node cost costs$[i]$.

The distance between any two nodes $i, j \in V$ is computed using the Euclidean metric rounded mathematically to the nearest integer:

$$D[i][j] = \left\lfloor \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} + 0.5 \right\rfloor$$

The symmetric distance matrix D and the 1D cost array costs are extracted immediately after loading the problem instance. In accordance with the problem specification, all optimization and evaluation functions access exclusively D and costs, without any reference to the node coordinates.

The goal is to select a subset of exactly $k = \lfloor (n+1)/2 \rfloor = \lceil 0.5 \times n \rceil$ distinct nodes and form a Hamiltonian cycle (closed tour) $\pi = (\pi_0, \pi_1, \ldots, \pi_{k-1})$ through the selected nodes such that the sum of the total tour length and the total cost of the selected nodes is minimized:

$$\min_{\pi} Z(\pi) = \sum_{i=0}^{k-1} \Big(D\big[\pi_i\big]\big[\pi_{(i+1) \bmod k}\big] + \text{costs}[\pi_i] \Big)$$

2 Pseudocode of Implemented Algorithms

In all greedy constructive heuristics below, 0-based indexing is used. For each greedy method, $n = 200$ solutions are generated by starting once from each node start $\in \{0, \ldots, n-1\}$. For the Random method, 200 independent random tours are evaluated.

2.1 Random Solution (RAND)

Algorithm 1: Random Solution Generation (`solve_random`)

Input: Number of nodes n, target tour size k, distance matrix D, cost vector costs
Output: Best random cycle π_{best} of length k

Initialize candidates $\leftarrow [0, 1, \ldots, n-1]$
for *trial* $\leftarrow 0$ **to** 199 **do**
  Shuffle candidates uniformly at random using Mersenne Twister (`mt19937`)
  cycle $\leftarrow$ candidates$[0 \ldots k-1]$
  score $\leftarrow$ evaluate_cycle(cycle, D, costs)
  Update min, max, sum, and π_{best}
end
return π_{best}

2.2 Nearest Neighbor – End Only ($\mathbf{NN_{end}}$)

Algorithm 2: Nearest Neighbor – Adding Only at the End (`solve_nn_end`)

Input: Number of nodes n, target size k, distance matrix D, cost vector costs, starting node start

Output: Constructed tour path of length k

```
1  visited[0 ... n − 1] ← false
2  visited[start] ← true
3  path ← [start]
4  while |path| < k do
5  |  best_node ← −1,   best_increase ← ∞
6  |  last ← path.back()
7  |  for v ← 0 to n − 1 do
8  |  |  if ¬visited[v] then
9  |  |  |  increase ← D[last][v] + costs[v]
10 |  |  |  if increase < best_increase then
11 |  |  |  |  best_increase ← increase
12 |  |  |  |  best_node ← v
13 |  |  |  end
14 |  |  end
15 |  end
16 |  visited[best_node] ← true
17 |  Append best_node to the end of path
18 end
19 return path
```

2.3 Nearest Neighbor – Any Position ($\mathbf{NN_{any}}$)

Algorithm 3: Nearest Neighbor – Adding at Any Position (`solve_nn_any`)

Input: Number of nodes n, target size k, distance matrix D, cost vector costs, starting node start

Output: Constructed tour path of length k

```
visited[0 ... n − 1] ← false
visited[start] ← true
path ← [start]
while |path| < k do
    cur_m ← |path|
    best_node ← −1,   best_pos ← −1,   best_increase ← ∞
    for v ← 0 to n − 1 do
        if ¬visited[v] then
            // 1. Consider inserting v at the beginning (index 0)
            d_0 ← D[v][path[0]] + costs[v]
            if d_0 < best_increase then
                best_increase ← d_0,   best_node ← v,   best_pos ← 0
            end
            // 2. Consider inserting v at the end (index cur_m)
            d_m ← D[path.back()][v] + costs[v]
            if d_m < best_increase then
                best_increase ← d_m,   best_node ← v,   best_pos ← cur_m
            end
            // 3. Consider inserting v inside path between path[i] and
               path[i + 1]
            for i ← 0 to cur_m − 2 do
                d_in ← D[path[i]][v] + D[v][path[i + 1]] − D[path[i]][path[i + 1]] + costs[v]
                if d_in < best_increase then
                    best_increase ← d_in,   best_node ← v,   best_pos ← i + 1
                end
            end
        end
    end
    visited[best_node] ← true
    Insert best_node into path at index best_pos
end
return path
```

2.4 Greedy Cycle (GC)

Algorithm 4: Greedy Cycle (`solve_greedy_cycle`)

Input: Number of nodes n, target size k, distance matrix D, cost vector costs, starting node start

Output: Constructed cycle cycle of length k

```
1  visited[0 ... n − 1] ← false
2  visited[start] ← true
3  cycle ← [start]
4  while |cycle| < k do
5  |  cur_m ← |cycle|
6  |  best_node ← −1,   best_pos ← −1,   best_increase ← ∞
7  |  for v ← 0 to n − 1 do
8  |  |  if ¬visited[v] then
9  |  |  |  for i ← 0 to cur_m − 1 do
10 |  |  |  |  u1 ← cycle[i]
11 |  |  |  |  u2 ← cycle[(i + 1) mod cur_m]
   |  |  |  |  // When cur_m = 1, u1 = u2 = start and D[u1][u2] = 0, giving
   |  |  |  |     2 · D[start][v] + costs[v]
12 |  |  |  |  increase ← D[u1][v] + D[v][u2] − D[u1][u2] + costs[v]
13 |  |  |  |  if increase < best_increase then
14 |  |  |  |  |  best_increase ← increase
15 |  |  |  |  |  best_node ← v
16 |  |  |  |  |  best_pos ← i + 1
17 |  |  |  |  end
18 |  |  |  end
19 |  |  end
20 |  end
21 |  visited[best_node] ← true
22 |  Insert best_node into cycle at index best_pos
23 end
24 return cycle
```

3 Results of Computational Experiments

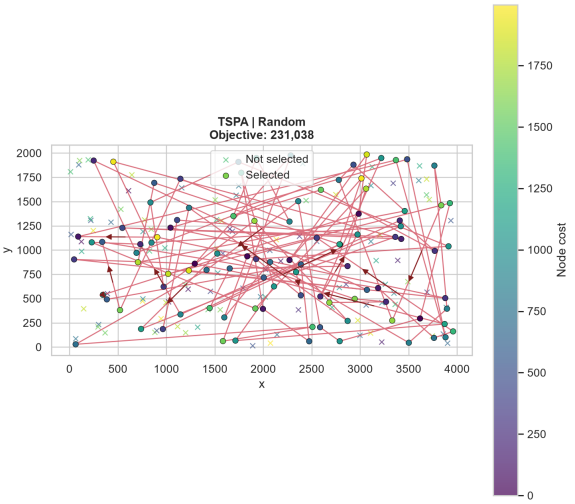
For each greedy heuristic, 200 solutions were generated by starting once from each node start $\in \{0, \ldots, 199\}$. For the Random method, 200 random solutions were generated. Table 1 summarizes the minimum, average, and maximum objective function values across both instances.

Table 1: Comparison of objective function values across instances `TSPA.csv` and `TSPB.csv`.

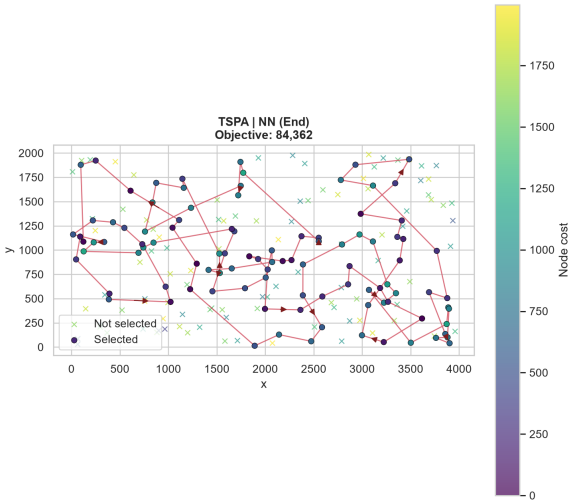
Instance	Method	Min	Max	Average	Rel. Gap to Best
TSPA.csv	Random (`solve_random`)	231038	287609	261318.44	+219.43%
	NN End (`solve_nn_end`)	84362	90075	87476.63	+16.64%
	NN Any Pos (`solve_nn_any`)	72328	77022	74346.97	0.00% (Best Min)
	Greedy Cycle (`solve_greedy_cycle`)	72590	75281	73793.40	+0.36% (Best Avg)
TSPB.csv	Random (`solve_random`)	181687	234674	205997.09	+243.36%
	NN End (`solve_nn_end`)	56007	63166	59397.33	+5.84% (Best Avg)
	NN Any Pos (`solve_nn_any`)	52915	66164	59480.27	0.00% (Best Min)
	Greedy Cycle (`solve_greedy_cycle`)	55655	63734	59649.91	+5.18%

4 2D Visualizations of the Best Solutions

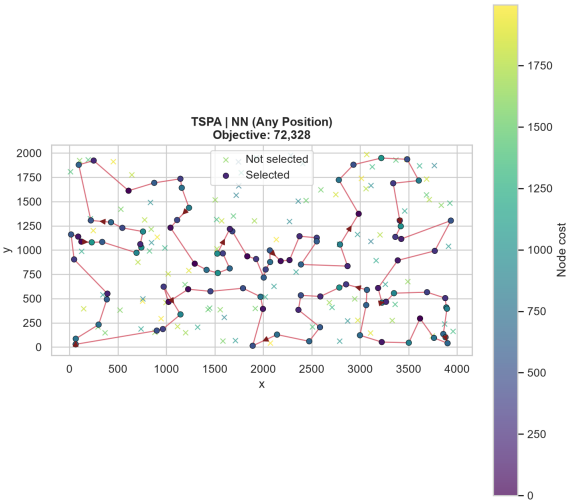
Figures 1 and 2 present the 2D visualizations of the best solutions obtained for each method on `TSPA.csv` and `TSPB.csv`. Node costs are encoded via node color/size.



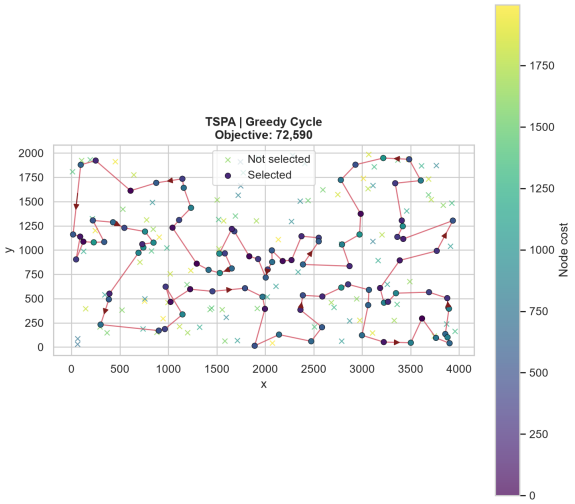
(a) Random (min = 231038)



(b) NN (End) (min = 84362)

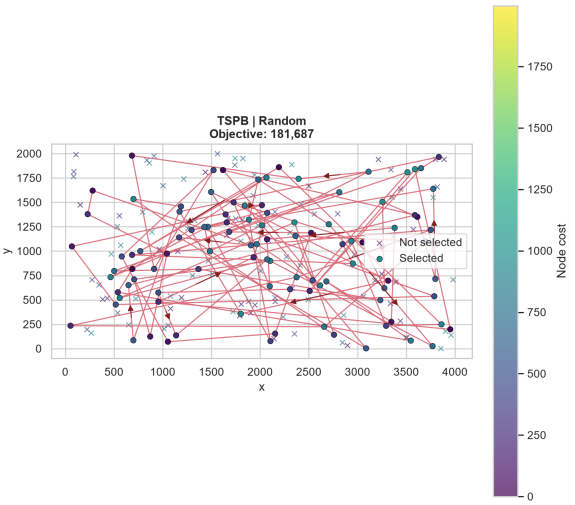


(c) NN (Any Pos) (min = 72328)

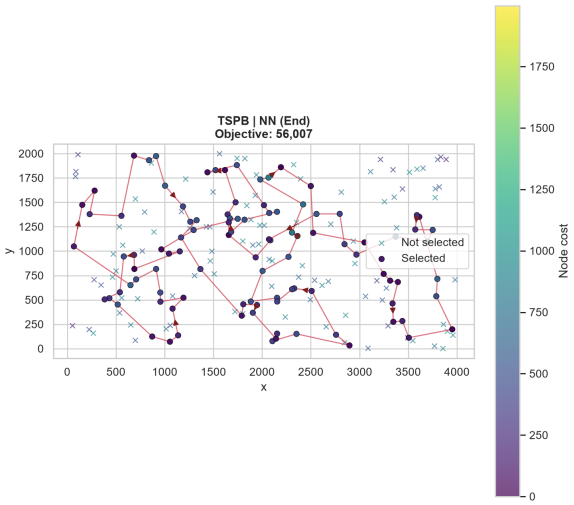


(d) Greedy Cycle (min = 72590)

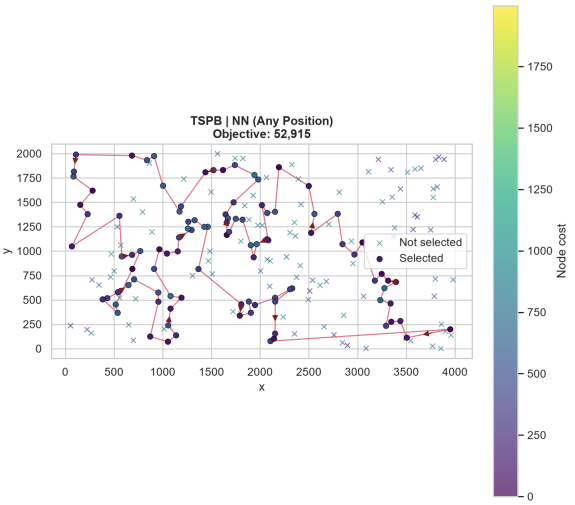
Figure 1: 2D Visualizations of the best solutions for **TSPA.csv**.



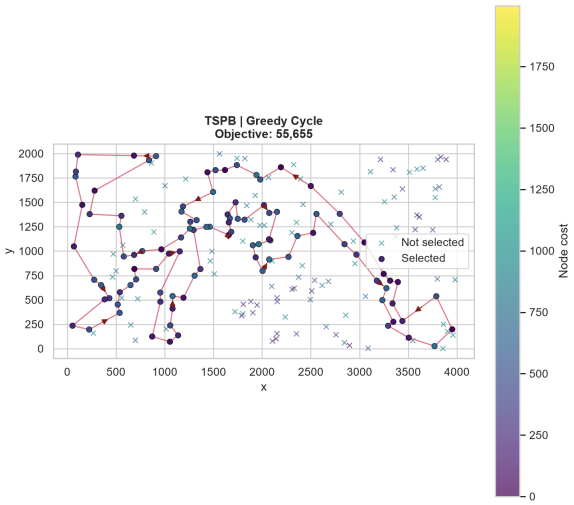
(a) Random (min = 181687)



(b) NN (End) (min = 56007)



(c) NN (Any Pos) (min = 52915)



(d) Greedy Cycle (min = 55655)

Figure 2: 2D Visualizations of the best solutions for **TSPB.csv**.

5 Best Solutions (Node Indices)

Below are the 0-indexed node lists printed by `print_cycle` for the best solution of each method.

5.1 Instance 1 (TSPA.csv)

- **Random:** 26,9,67,98,179,64,170,55,6,124,186,136,59,12,91,8,189,123,11,121,56,13,89,0,156,92,169,90,126,50,152,3,99,84,144,37,115,38,30,190,138,4,2,194,195,36,197,164,95,127,131,192,135,5,41,25,75,140,180,168,150,48,160,163,134,142,76,23,21,107,111,158,62,28,104,20,22,39,191,159,68,149,19,193,29,132,174,185,27,1,47,34,133,175,143,157,120,49,106,88
- **NN (End):** 54,9,33,36,177,67,144,53,135,58,134,176,174,106,89,107,57,182,151,93,50,3,1,164,178,29,136,186,184,47,159,130,153,78,88,193,170,146,75,64,150,42,113,43,85,142,8,131,84,158,192,181,114,141,180,132,76,80,48,31,63,149,168,97,127,187,30,123,152,108,60,116,49,11,119,160,83,90,51,16,35,87,45,140,52,179,185,28,102,156,157,71,91,38,100,4,112,139,115,5

- **NN (Any Pos):** 104,129,69,127,97,168,116,149,63,11,49,160,83,123,100,152,108,60,31,48,80,91,157,71,4,76,132,180,28,102,185,35,156,51,16,67,144,140,53,179,52,177,36,33,9,54,131,174,8,164,186,136,163,29,101,178,1,169,3,50,147,47,93,106,42,89,113,130,159,146,170,153,193,78,88,107,64,57,182,151,150,176,72,134,84,58,158,192,181,114,135,73,45,87,141,187,30,189,2,171
- **Greedy Cycle:** 31,116,149,168,63,11,49,60,108,152,100,90,123,83,160,97,127,69,171,2,189,30,187,141,87,45,73,135,114,181,192,158,58,84,134,72,176,150,151,182,57,64,107,88,78,193,153,170,146,159,130,113,89,42,106,93,47,147,50,3,169,1,178,101,29,163,136,186,164,142,8,174,131,54,9,33,36,177,52,179,53,140,144,67,16,51,156,35,185,102,28,180,132,76,4,71,157,91,80,48

5.2 Instance 2 (TSPB.csv)

- **Random:** 92,72,65,197,118,172,189,124,194,83,116,97,198,177,134,179,76,160,199,66,135,143,163,1,11,148,125,106,162,159,74,68,89,26,32,170,6,9,84,20,41,184,25,120,54,52,12,114,56,139,21,77,19,64,182,34,155,45,111,5,0,99,123,130,62,180,196,96,119,173,133,63,129,101,191,94,169,67,165,110,71,151,46,102,168,82,95,156,144,93,22,115,175,140,145,149,122,38,17,137
- **NN (End):** 32,89,57,76,54,127,129,90,153,20,14,2,110,118,56,114,196,10,148,143,78,192,162,104,185,184,3,151,69,172,133,120,96,121,198,48,72,46,86,117,160,136,8,21,154,140,47,170,125,177,6,132,29,197,175,134,45,60,40,71,155,97,115,180,65,28,149,145,158,116,147,42,50,130,30,183,38,64,85,59,167,51,87,179,82,88,171,99,67,24,34,181,152,73,98,27,135,75,79,156
- **NN (Any Pos):** 110,2,14,20,74,153,182,52,90,129,127,80,54,192,162,104,76,185,57,89,86,46,72,3,184,173,151,122,117,8,160,69,172,136,133,120,156,43,48,96,198,121,130,191,50,42,147,116,37,146,13,149,28,145,65,158,177,125,26,170,197,36,132,29,6,180,115,155,71,175,134,45,157,60,40,141,97,154,140,47,38,21,176,30,183,189,168,64,59,85,51,73,167,152,34,24,181,179,87,82
- **Greedy Cycle:** 174,36,197,6,180,115,170,97,140,47,155,71,175,134,45,157,60,141,40,33,64,21,189,168,69,160,172,136,133,120,8,117,72,46,86,184,3,173,122,151,98,111,27,135,76,185,162,192,80,52,182,20,74,2,93,110,118,14,153,90,129,127,54,104,57,89,156,43,48,96,198,121,63,130,191,183,30,176,38,154,26,125,177,138,158,145,149,147,42,116,37,146,13,28,65,178,103,29,132,1

6 Verification with Solution Checker

All best solutions presented in Section 5 were verified using the official solution checker. For both `TSPA.csv` and `TSPB.csv`, the objective function values computed by `evaluate_cycle` matched the solution checker output with zero discrepancy.

7 Source Code

The complete C++ implementation is available at:
https://github.com/fpodalak/Evolutionary-Computation

8 Conclusions

Based on the computational experiments, we formulate the following conclusions:

1. **Greedy Heuristics vs. Random Baseline:** All three greedy heuristics (`solve_nn_end`, `solve_nn_any`, and `solve_greedy_cycle`) substantially outperform the Random baseline across minimum, mean, and maximum objective values.

2. **Effect of Insertion Position Flexibility ($\mathbf{NN_{end}}$ vs. $\mathbf{NN_{any}}$):** Allowing insertions at any position along the open path (`solve_nn_any`) rather than restricting additions strictly to the end (`solve_nn_end`) significantly improves solution quality. In $\mathrm{NN}_{\mathrm{end}}$, the algorithm only looks at $D[\mathrm{last}][v] + \mathrm{costs}[v]$ and is often forced into long jumps or high-cost nodes late in the path, plus a potentially long return edge $D[\mathrm{path}[k-1]][\mathrm{path}[0]]$. By contrast, $\mathrm{NN}_{\mathrm{any}}$ can insert cheap nodes into the middle or front of the path wherever they fit with minimal detour.

3. **Open Path (`solve_nn_any`) vs. Closed Cycle (`solve_greedy_cycle`):** While `solve_nn_any` constructs an open path and only accounts for the closing edge $D[\mathrm{path}[k-1]][\mathrm{path}[0]]$ during final evaluation, `solve_greedy_cycle` maintains a valid closed cycle at every iteration. On both `TSPA.csv` and `TSPB.csv`, `solve_nn_any` achieved the lowest overall minimum objective value (72328 and 52915, respectively), whereas `solve_greedy_cycle` achieved the best average performance on `TSPA.csv` (73793.40).